

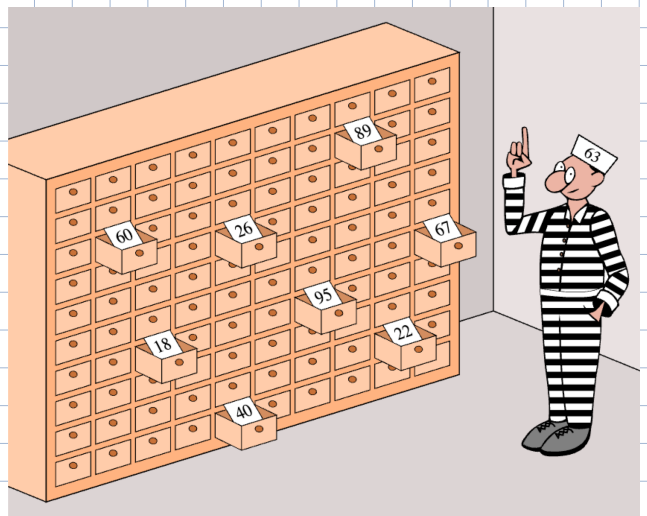
100 Prisoners Problem

The director of a prison offers a last chance of freedom to 100 prisoners.
scientists

Trump assigns a distinct number in $\{1, \dots, 100\}$ to each scientist

There is a room with 100 drawers.
Each containing a number in $\{1, \dots, 100\}$

Now, each scientist enters the room alone. (S) He can open up to 50 drawers.
He wins if he finds his number.
His number.



They win if all of them find their numbers

Scientists cannot communicate but they may have agreed on a strategy

Trivial Strategy

Each scientist chooses 50 drawers uniformly at random

$$P(\text{a scientist wins}) = \frac{50}{100} = \frac{1}{2}$$

$$P(\text{all win}) = \frac{1}{2^{100}} \approx 0.\underbrace{00 \dots 0}_{33}8$$

$$P(\text{all win}) \approx 0.3$$

- Ob, Scientists does not need to choose drawers before hand
She can use an adaptive strategy
How?

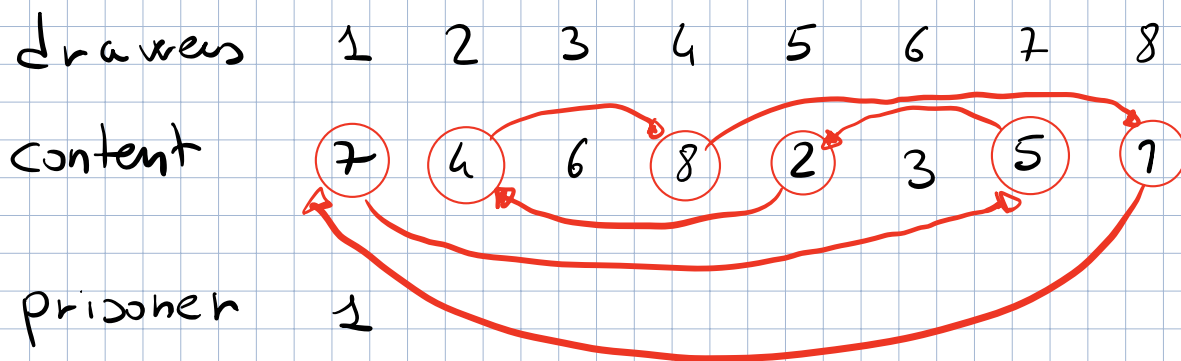
Not only scientists are numbered but also drawers e.g. Row wise from left to right

The strategy is as follows

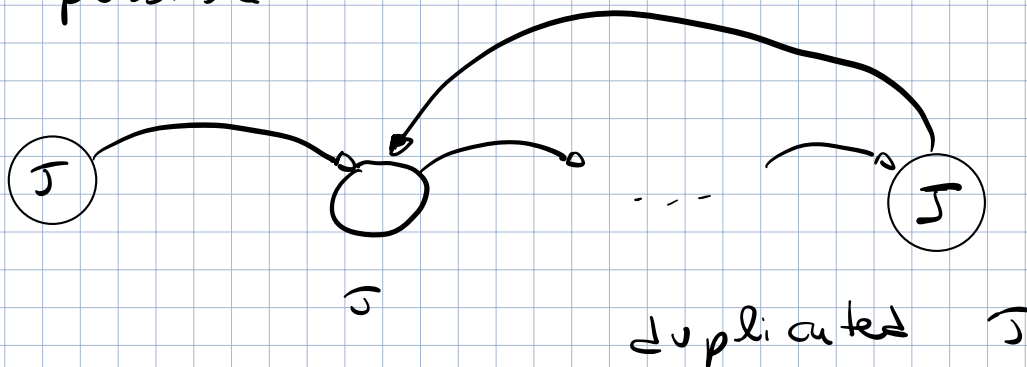
- Scientist number i opens drawers i with number d_i
- If $d_i = i$, he wins

- otherwise, he opens drawer d_i and continue

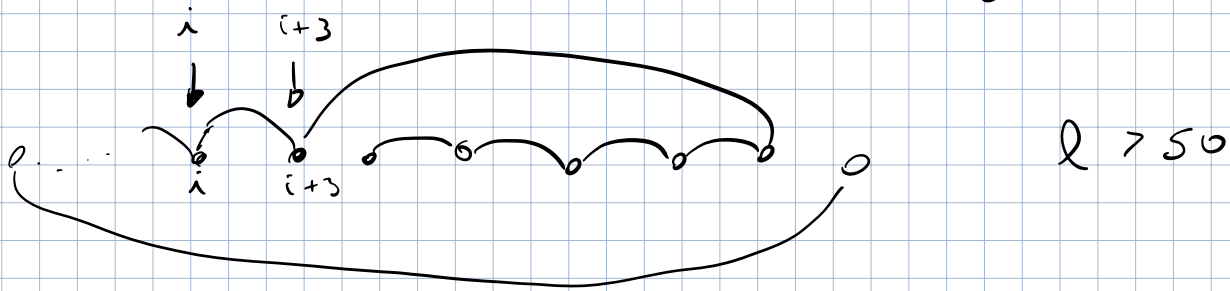
Obs. if we do not stop after 50, the scientist is guaranteed to find his number



not possible



$$P(\text{All win}) = P(\text{random permutation has no cycle longer than 50})$$

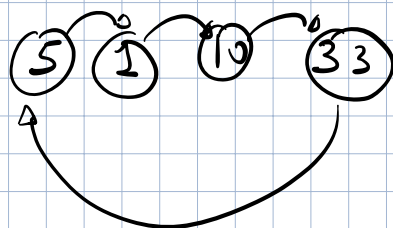
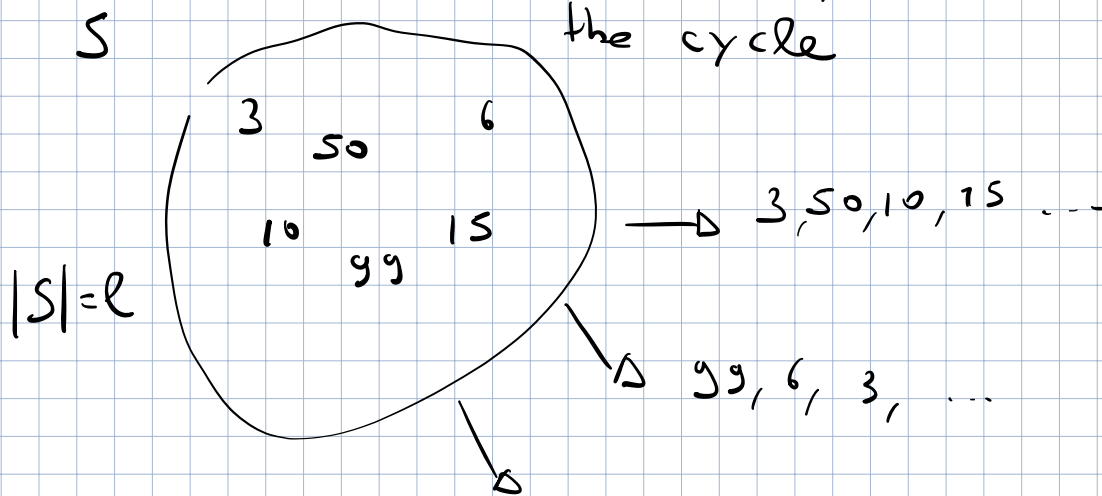


Number of permutations ^(of 100) with a cycle of length $l > 50$

for $l > 50$, only one cycle is possible

$$\binom{100}{l} \cdot (l-1)! \cdot (100-l)! = \frac{100!}{l}$$

$\uparrow$ elements of the cycle $\uparrow$ their order



$$\rightarrow 5, 1, 10, 33 \dots$$

|||

$$33, 5, 1, 10, \dots$$

Total number of permutation with a cycle
of length > 50

$$\begin{aligned}\sum_{l=51}^{100} \frac{100!}{l} &= 100! \sum_{l=51}^{100} \frac{1}{l} \\&= 100! \cdot \left(\frac{1}{51} + \frac{1}{52} + \frac{1}{53} + \dots + \frac{1}{100} \right) \\&= \underline{100! \cdot (H_{100} - H_{50})} \quad \begin{array}{l} \Delta \text{ } n\text{th Harmonic} \\ \text{number} \end{array}\end{aligned}$$

$$P(\text{win}) = 1 - (H_{100} - H_{50}) \approx 0.3118$$

Duplicate elements in an array

We are given an array $A[0, \dots, n]$ of $n+1$ numbers in $\{0, \dots, n-1\}$

By pigeonholes principle, there must be at least a duplicated elements (e)

Find one duplicate

	0	1	2	3	4	5	6	7	8	
A	1	3	5	6	5	2	0	4	7	$e = 5$

Solution 1: Use a Hash Set H scan from left to right and stop when current e is already in H

Solution 2:

Direct Access Table

B	0	1	2				k		j		$n-1$
	0	0	0				1		0		0
							↑				

Complexity: $\Theta(n)$ time but $\Theta(n)$ extra space (frowning face)

Constant extra space?

It seems to be impossible because we believe we need to keep track of the element we have seen so far.

$\Theta(\log n)$ passes over A

$\Theta(\log n)$ passes to "reconstruct" e
bit by bit

	0	1	2	3	4	5	6	7	8
A	1	3	5	6	5	2	0	4	7

$\rightarrow$	0	0	1	1	1	0	0	1	1
$\rightarrow$	0	1	<u>0</u>	<u>1</u>	<u>0</u>	<u>1</u>	<u>0</u>	<u>0</u>	<u>1</u>
	1	1	<u>1</u>	0	<u>1</u>	0	0	<u>0</u>	<u>1</u>

e

1
0
1
||
5

Pass #1

0	1
4	5

Pass #2

0	1
<u>3</u>	2

restrict to numbers starting
with 1

Pass #3

0	1
1	2

only numbers starting with 10

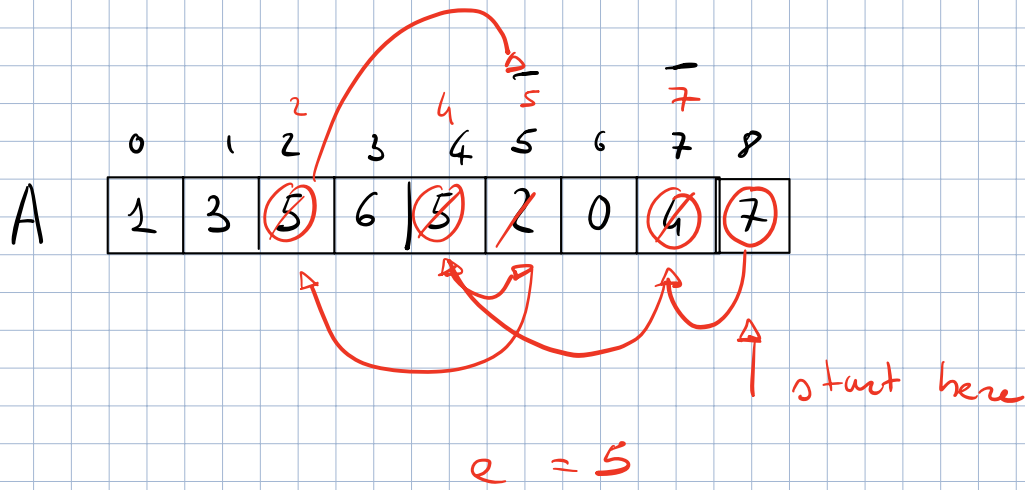
$\Theta(n \log n)$ time

$O(1)$ space

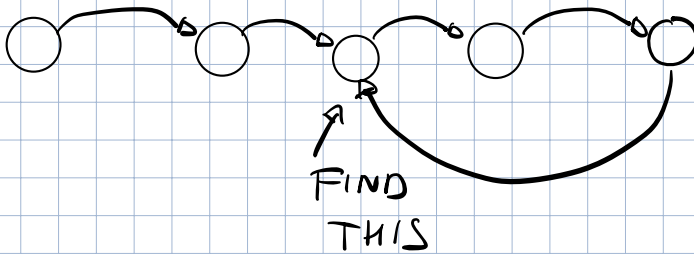
BEST IF NO RANDOM ACCESS

DOES RANDOM ACCESS HELP?

DESTROY A



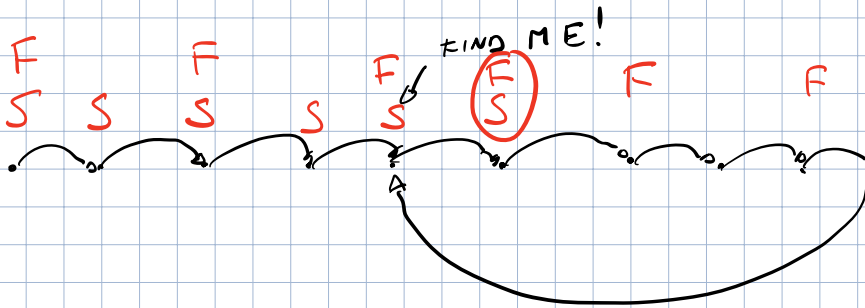
$\Theta(n)$ time $O(1)$ but destroying A $\hat{=}$



Floyd's cycle finding

Find the beginning of a cycle in a list in $O(1)$ extra space.

The list is READ-ONLY.



- Use two pointers

S	Slow	moves just 1 step
F	Fast	moves 2 steps

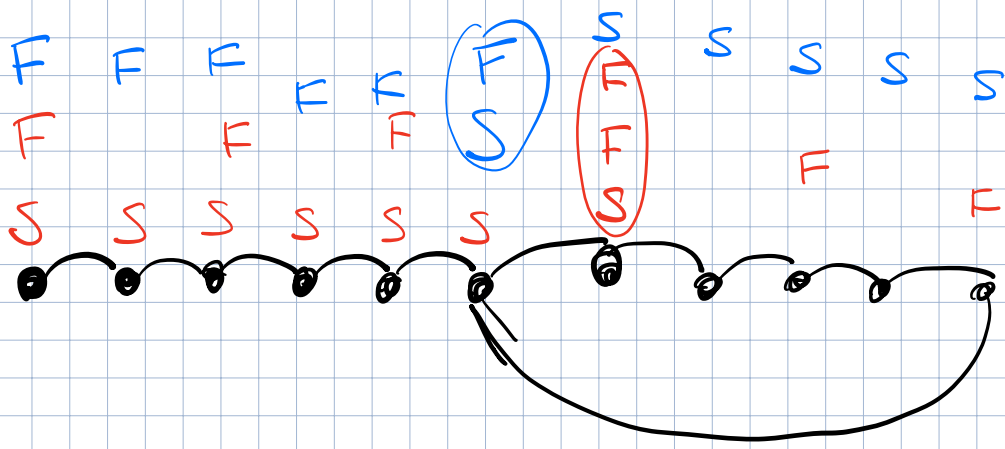
Phase #1

- First, they meet in the cycle, certifying a cycle exists

Phase #2

- Move F at the begin and set its speed to 1
- Magic: they'll meet again at the begin of the cycle!!!

A simple hand-drawn smiley face on a blue grid background. It consists of two vertical lines for eyes and a wide, upward-curving line for a mouth.



Majority Element

We are given an array $A[1, n]$.

Find, if any, the element that occurs at least $\frac{n}{2} + 1$ times

Solutions

- Sort $\Theta(n \log n)$ time $\Theta(n)$ space
- HashMap $\Theta(n)^*$ time $\Theta(n)$ space

Is $\Theta(n)$ time and $O(1)$ space possible?

Boyer-Moore Algorithm

A

5	3	1	1	2	3	1	1	1
---	---	---	---	---	---	---	---	---

- keep a candidate c and a counter
- process the array from left to right
 - if $A[i] == c$
 - increase the counter by 1
 - otherwise
 - decrease the counter by 1
- if counter == 0
 - $c = A[i]$
 - set counter to 1

A

5	3	1	1	2	3	1	1	1
---	---	---	---	---	---	---	---	---

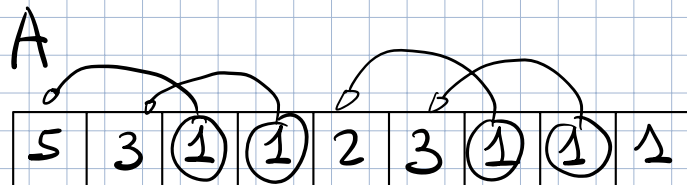
C 5 3 1 1 1 3 1 1 1
 counter 1 1 1 2 1 1 1 2 3

is not the frequency of 1

CLAIM

if there is a Majority element
 then at the end $c = e$

why?



we can use an occurrence of
 the majority element to "kill"
 another element

Support Insertion / Deletion?

A

5	3	1	1	2	3	1	1	1
---	---	---	---	---	---	---	---	---

1 0 0 0 0 0 0 0 0

0 1 0 0 1 1 0 0 0

1 1 1 1 0 1 1 1 1

0	1	MAY
8	2	0
6	3	0
1	8	1

1

$\Theta(\log n)$ time and space

D D D D D I

1 1 1 1 1 3

u is the largest element

0	1	
4	8	2
1	6	3 4
1	8	3 4

3

Misra - Gries Heavy Hitter

Given $A[1, n]$, find the T -elements that occur at least $\frac{n}{T} + 1$ times

keep a set k of T candidates with counters

for e in A :

if e in k

$k[e] += 1$

else

$k[e] = 1$

if $\text{len}(k) > T$:

for k in $k.\text{keys}()$

$k[k] -= 1$

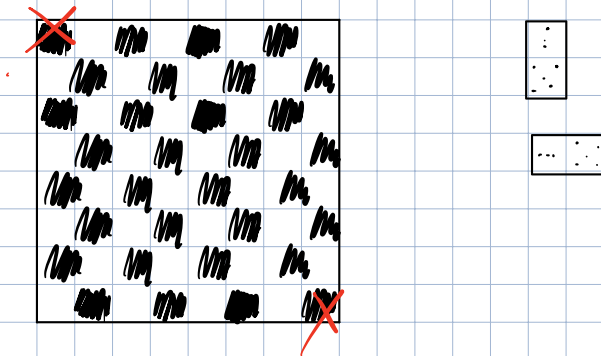
if $k[k] == 0$:

$\text{del}(k[k])$

Running time!

- NAIVE ANALYSIS: $O(T \cdot n)$ time

but actually is $\Theta(n)$ because
the cost of the for k in k loop
cannot be more than the number
of element of the array.



Is it possible to cover this chess board,
with dominos?